

Troubleshooting an EKS Cluster That Is Not Scaling Properly

Scenario:

An Amazon EKS cluster is not scaling up properly when application demand increases. The goal is to identify the root cause and restore automatic scaling.

Step 1: Verify Pod Status

Check whether pods are stuck in Pending state using: `kubectl get pods -A`

If pods remain pending, inspect them using: `kubectl describe pod <pod-name>`

Look for resource shortages, node selector issues, taints, or scheduling errors.

Step 2: Check Cluster Autoscaler

Verify that Cluster Autoscaler is running: `kubectl get pods -n kube-system`

Review logs: `kubectl logs <autoscaler-pod> -n kube-system`

Common issues include IAM permission problems and incorrect node group configuration.

Step 3: Verify Node Group Configuration

Check EKS Managed Node Groups or Auto Scaling Groups. Confirm: Minimum, desired, and maximum node counts are configured correctly. The maximum size has not been reached. Nodes can launch successfully. **Step 4: Check Resource Availability**

Use: `kubectl top nodes`

Verify CPU, memory, and storage availability. Pods cannot schedule if nodes lack sufficient resources.

Step 5: Validate HPA Configuration

Check Horizontal Pod Autoscaler: `kubectl get hpa`

Verify target CPU or memory metrics and ensure metrics are being collected properly.

Step 6: Verify Metrics Server

Ensure Metrics Server is running: `kubectl get deployment metrics-server -n kube-system`

Without metrics, HPA cannot make scaling decisions.

Step 7: Check AWS Limits and Capacity

Review: EC2 instance quotas Subnet IP availability Availability Zone capacity Auto Scaling Group events **Step 8: Review Events and Logs**

Use: `kubectl get events --sort-by=.metadata.creationTimestamp`

Check CloudWatch logs, EKS control plane logs, and Cluster Autoscaler logs for errors.

Common Causes

- Cluster Autoscaler misconfiguration
- HPA not receiving metrics
- Node group maximum size reached
- Insufficient EC2 capacity
- IAM permission issues
- Resource requests set too high
- Subnet IP exhaustion

Resolution

After identifying the root cause, update node group limits, fix IAM permissions, deploy Metrics Server, adjust resource requests, or expand subnet capacity. Validate scaling by generating load and monitoring pod and node growth.

Conclusion

Troubleshooting EKS scaling issues requires checking Pods, HPA, Cluster Autoscaler, Node Groups, Metrics Server, AWS quotas, and logs. A systematic approach helps quickly identify and resolve scaling bottlenecks.